Рекуррентные нейронные сети

Рекуррентные нейронные сети (RNN) — это тип нейронных сетей, предназначенных для обработки последовательных данных, где важен порядок элементов и контекст предыдущих шагов. В отличие от обычных нейросетей, которые принимают входы независимо, RNN имеют внутреннюю память (скрытое состояние), накапливающую информацию о предыдущих элементах последовательности.

Принцип работы RNN основан на применении рекуррентных связей: выход скрытого слоя одного временного шага подается обратно в этот же слой как вход на следующий шаг. Это позволяет сети учитывать прошлый контекст при обработке текущего входа, что особенно важно для задач обработки текста, речи, временных рядов.

На каждом временном шаге RNN принимает входной вектор и предыдущее скрытое состояние, вычисляет новое скрытое состояние, использует его для предсказания и передает дальше. Обучение происходит с помощью алгоритма обратного распространения ошибки сквозь время (BPTT), где ошибка распространяется по всем временным шагам.

RNN хорошо подходят для задач:

Машинный перевод,

Распознавание речи,

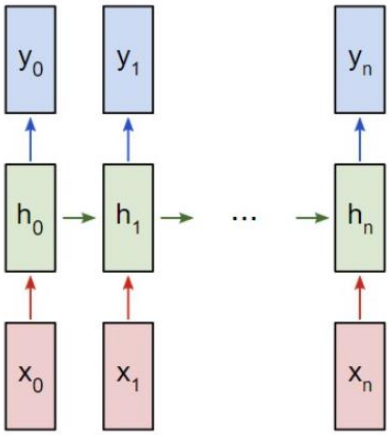
Генерация текста,

Анализ временных рядов.

Однако простые RNN могут испытывать трудности с долгосрочной зависимостью из-за проблем затухающего градиента, что ведет к разработке более сложных архитектур, таких как LSTM и GRU.

Архитектура Рекуррентных нейронных сетей

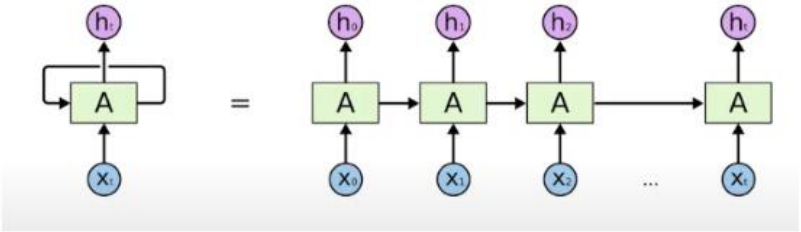
Несмотря на то, что RNN могут хорошо справляться со своими задачами они не могут работать с длинными последовательностями. Эффективно они могут работать только с последовательностями состоящими из 3-4 элементов. Для, к примеру, анализа текста отзывов на предмет того положительный это отзыв или нет этого будет недостаточно. Здесь может понадобиться анализ нескольких десятков слов, чтобы сделать корректный вывод.



Типичная классическая рекуррентная нейронная сеть использует только три набора параметров веса для выполнения требуемых подсчетов:

1. W_{xh} используется для всех связок $x_t \rightarrow h_t$
2. W_{hh} используется для всех связок $h_{t-1} \rightarrow h_t$
3. W_{hy} используется для всех связок $h_t \rightarrow y_t$

$$h_t = tanh(W_{xh}x_t + W_{hh}h_{t-1})$$
$$y_t = W_{hy}h_t$$



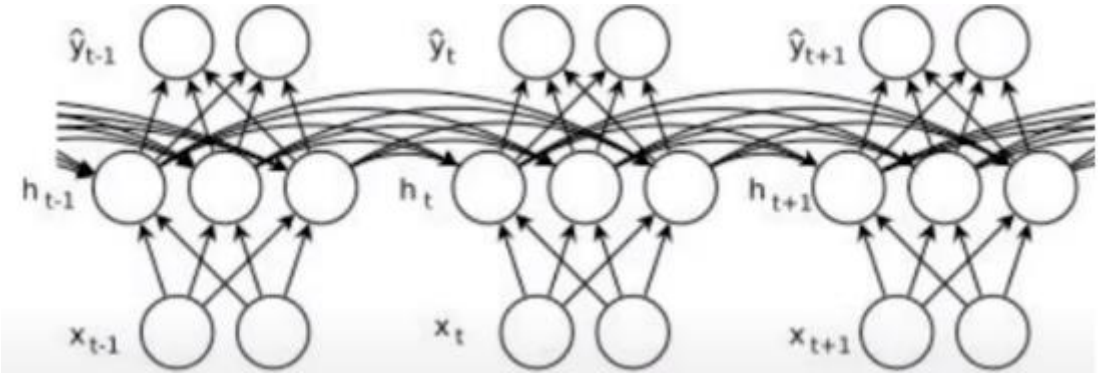
До этого в одном нейроне функция активации была:

$$h = tanh(Wx)$$

А теперь в нейроне с обратной связью:

$$h = tanh(W_1h + W_2x)$$

Вот так можно проиллюстрировать связи в нейронах RNN:



Выход каждого нейрона подключен ко входам всех нейронов.

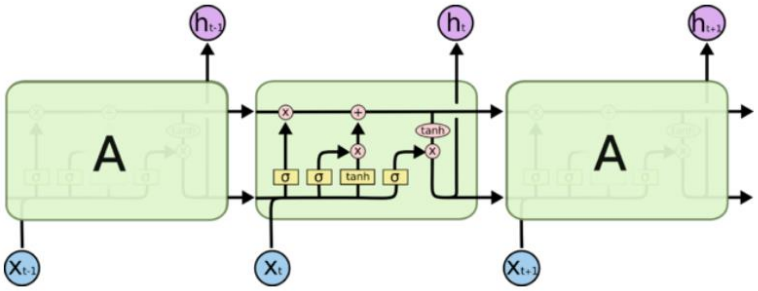
Из материалов по сверточным нейронным сетям известна проблема исчезающего градиента. В случае с большим количеством слоев значение

градиента при последовательном обновлении большого количества слоев становится все меньше и может стать настолько маленьким, что не сможет в принципе существенно изменить поведение нейронов. В рекуррентных нейронных сетях из-за сигнала циркулирующего внутри слоев это проблема становится еще острее. Причем градиент может стать не только очень маленьким, но и очень большим.

Long Short Term Memory (LSTM)

Решить проблему исчезающего градиента призвана разновидность RNN под названием LSTM.

Long short-term memory (LSTM) юниты - это блоки, из которых состоят слои одной из разновидностей рекуррентной нейронной сети (RNN). Обычно LSTM юнит представляет из себя ячейку состоящую из input gate, output gate и forget gate. Эти ячейки ответственны за запоминания значений на определенные промежутки времени. RNN, состоящая из LSTM юнитов иногда называется просто LSTM.



Здесь не просто распространяется сигнал, но и происходит управление размером влияния того, что приходит из предыдущего состояния. Каждый из этих элементов можно представить как типичный искусственный нейрон, который вычисляет активацию как взвешенную сумму. Их работа сводится к регуляции потока значений через блок LSTM, поэтому они и называются ворота или затворы (gate). Задача gate'ов - открыть пошире двери или закрыть её. Есть три gate:

Input gate - gate входного сигнала обеспечивает возможность меньше влиять предыдущему состоянию на текущее состояние

Output gate - gate выходного сигнала

Forget gate - gate забывания управляет очисткой памяти, скоростью забывания предыдущего состояния. Хранить эту информацию данная ячейка может очень долго.

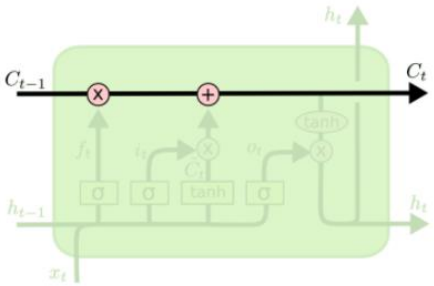
Из-за этих настраиваемых параметров, LSTM более сложная модель по сравнению с RNN. Значит она требует больше времени, больше примеров.

Понятие долгой памяти в названии возникло из-за того, что они могут запоминать информацию на более длинный период времени, чем обычная RNN. LSTM хорошо подходит для классификации процессов и предсказания временных последовательностей неизвестного размера и неизвестных промежутков между

важными событиями. С технической точки зрения это достигается за счет ликвидации проблем связанных с exploding и vanishing gradient'ами. Ошибки сохраняются в самом слое, что позволяет передавать градиенты в следующие слои практически без потерь.

Компоненты LSTM.

Ключевой момент в LSTM - это состояние ячейки - горизонтальная линия сверху диаграммы. Сигнал протекает из ячейки в ячейку с небольшими линейными преобразования.



У LSTM есть способность удалять или добавлять информацию в состояние ячейки с помощью gate'ов.

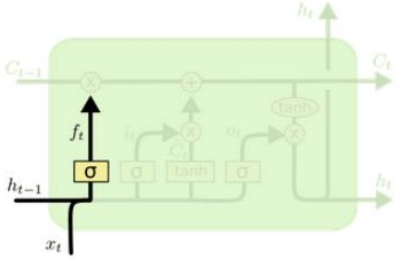
Гейты пропускают информацию по возможности. Они состоят из сигмоиды и перемножения матриц. Выход с сигмоиды от 0 до 1 описывает с какой силой сигнал может пройти дальше. Значения 0 значит "ничего не пройдет", а 1 - "пропускай всё".

У LSTM есть три гейта, чтобы сохранять и контролировать состояние ячейки.

Forget gate

Первый шаг в LSTM - это решить, от какой информации мы хотим избавиться. Это решение принимает слой с сигмоидой, который называется "forget gate layer." (гейт забывания). Он принимает во внимание h_{t-1} и x_t , а на выходе даёт значение между 0 и 1 для каждого числа в состоянии ячейки C_{t-1} . 1 значит "полностью сохрани это", а 0 - "полностью забудь это".

Пример забывания - языковая модель пытается предсказать следующее слово базируясь на предыдущих. Здесь модель может запоминать род объекта, чтобы использовать правильное образование слов. Когда мы видим новый объект, то нужно забыть род предыдущего объекта. (Животное не переходило дорогу, потому что оно устало)

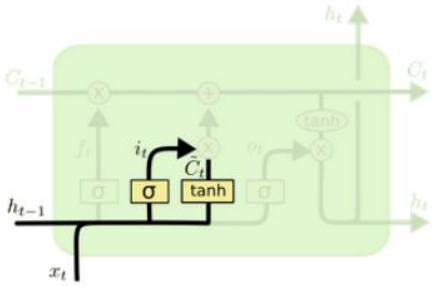


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Input gate

Следующий шаг - решить, какую информацию мы должны хранить в состоянии ячейки. Шаг состоит из двух частей. Первая - слой сигмоиды, называемый "input gate layer" (входной гейт), который решает какие значения будут обновляться. Вторая - слой с тангенсом, который создает вектор значений C~t , которые будут добавляться к состоянию ячейки.

С примером языковой модели, мы бы хотели добавлять род нового объекта в состояние ячейки, чтобы заменить старый род, который мы забудем.



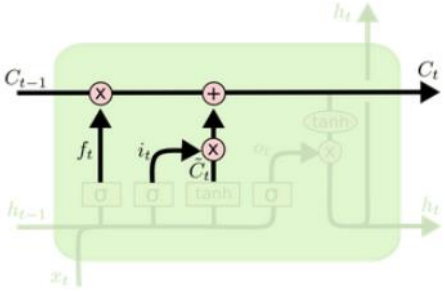
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Update cell state

Сейчас самое время, чтобы обновить старое состояние Ct-1 в новое состояние Ct. Предыдущие шаги уже решили, что делать, нужно только сделать это.

Умножаем старое состояние на ft, тем самым забывая те вещи, которые хотели забыть, затем прибавляем it∗Ct. Это новое значение состояния ячейки, которое отмасштабировано в зависимости от того, насколько мы хотим обновить новое значение.

В языковой модели, это момент, где мы выкидываем информацию о роде старого объекта и добавляем новую информацию о роде нового объекта.

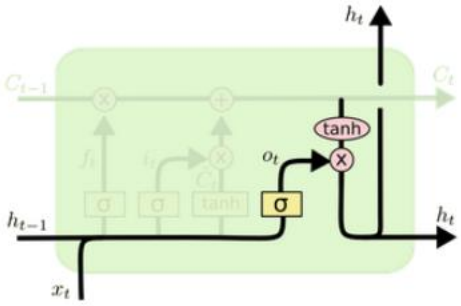


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output gate

Наконец-то нам нужно решить, что мы отправим на выход. Выход будет базироваться на состоянии ячейки, но с небольшой фильтрацией. Во-первых, прогоним входной сигнал через сигмоиду, которая решает с какой силой дальше пропускать сигнал, во-вторых, прогоняем состояние ячейки через тангенс и умножаем это на сигмоиду, чтобы пропускать дальше только то, что мы решили пропустить.

Для языковой модели, которая видит только объект, здесь можем пропустить информацию, связанную с глаголом. Например, на выходе может быть полезно число множественной или единственное у объекта, чтобы знать в какую форму нужно поставить глагол.



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

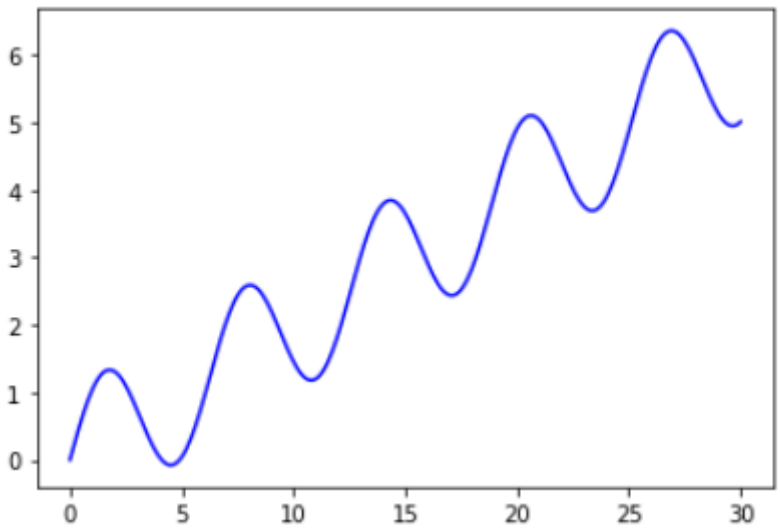
$$h_t = o_t * \tanh\left(C_t\right)$$

Пример. Возьмем последовательность данных - сунисоиду

```
from tensorflow.keras.layers import Input, LSTM, Dense, Flatten,
Dropout
from tensorflow.keras.models import Model
import matplotlib.pyplot as plt
import numpy as np
N = 300
t = np.linspace(0, 30, N).reshape((N, 1))
x_t = np.sin(t) + t * 0.2

x_len = 7  # длина истории
n = 2  # длина вектора данных

plt.plot(t, x_t, '-b');
```




```
input = Input(shape=(x_len, 2))
output = LSTM(units=128, dropout=0.0)(input)
output = Dense(1, activation='linear')(output)

model = Model(input, output)
model.compile(optimizer='Adam',
              loss='mse',
              metrics=['mse'])

model.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 7, 2)]            0

 lstm (LSTM)                 (None, 128)               67072

 dense (Dense)               (None, 1)                 129

=================================================================
Total params: 67,201
Trainable params: 67,201
Non-trainable params: 0
```

```
X = []
y = []

for i in range(N - x_len - 1):
    X.append(np.concatenate(
                (t[i: x_len + i], x_t[i: x_len + i])
        ).reshape((2, x_len)).T
    )
    y.append(x_t[i + x_len])

X = np.array(X)
y = np.array(y)
```

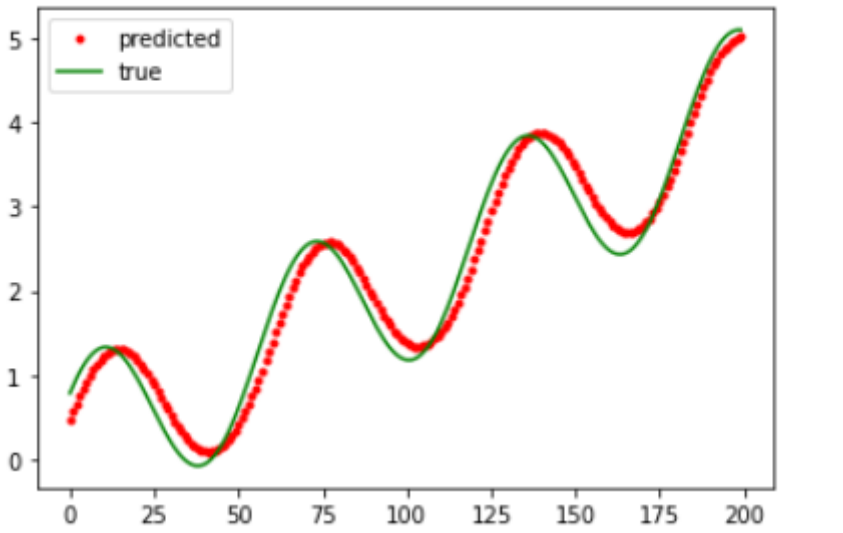
```
%%time
test_n = 200
model.fit(X[:test_n], y[:test_n],
          epochs=20,
          validation_data=(X[test_n:], y[test_n:])
          )
```

```
Epoch 1/20
7/7 [==============================] - 5s 79ms/step - loss: 5.7271 - mse: 5.7271 - val_loss: 15.8870 - val_mse: 15.8870
Epoch 2/20
7/7 [==============================] - 0s 8ms/step - loss: 1.5474 - mse: 1.5474 - val_loss: 6.7048 - val_mse: 6.7048
Epoch 3/20
7/7 [==============================] - 0s 8ms/step - loss: 1.4215 - mse: 1.4215 - val_loss: 5.2635 - val_mse: 5.2635
Epoch 4/20
7/7 [==============================] - 0s 8ms/step - loss: 0.7541 - mse: 0.7541 - val_loss: 5.6836 - val_mse: 5.6836
```

```
model.evaluate(X[:test_n], y[:test_n])
y_pred = model.predict(X[:test_n])

plt.plot(y_pred, '.r', label='predicted')
plt.plot(y[:test_n], 'g', label='true')
plt.legend();
```

```
7/7 [==============================] - 0s 2ms/step
```




```
y_pred = model.predict(X[test_n:])

plt.plot(y_pred, '.r', label='predicted')
plt.plot(y[test_n:], 'g', label='true')
plt.legend();
```

```
3/3 [==============================] - 0s 4ms/step
```

